DOCKER · LABORATORY ACTIVITY

Containerize It

Package the SecureNotes API into an image, then run the whole stack with Compose

Course	Units	Work in	Est. time	Submit
Web Systems and Technologies	1 (Laboratory)	Pairs	2–3 hours	GitHub repo + screenshots

The scenario. SecureNotes is built, secured, and tested — it runs fine on your laptop. But "runs on my laptop" isn't shipping. In this lab you **package the app into a Docker image** so it runs the same on any machine, then use **docker-compose** to bring it up next to a real Postgres database with one command. You write three files; the app and its tests stay exactly as given.

What you'll practice

Build & run an image

- writing a Dockerfile for a Python API
- layer caching (why order matters)
- a small, pinned, non-root image
- build, run, and map a port

Run the whole stack

- a .dockerignore for a clean build
- docker-compose with two services
- containers talking over a network
- a volume so data survives

Setup (10 minutes)

```
# the app runs the same as before
python -m venv venv
source venv/bin/activate   # Win:
venv\Scripts\activate
pip install -r requirements.txt
pytest -q                  # 12 app tests pass;
15 Docker checks red

# Docker must be installed and running
docker --version
docker run --rm hello-world
```

You need Docker Desktop installed and running, and internet the first time (Docker downloads the base images).

You write three files: `Dockerfile`, `.dockerignore`, `docker-compose.yml`. Don't edit the app or the tests. **Keep the bcrypt pin** (`bcrypt==4.0.1`).

Two structural test files guide you — they read your files and check the structure, but can't build the image (that's real Docker on your machine): `pytest tests/test_dockerfile.py` for Parts A–B, `pytest tests/test_compose.py` for Part C.

Part A — Write the Dockerfile (~40 min)

Fill in each step in `Dockerfile`. The order matters — it's what keeps rebuilds fast.

Task	Add	Why
A1	`FROM python:3.12-slim`	small, pinned, official base — not `:latest`.
A2	`WORKDIR /app`	a home for the app inside the image.
A3	`COPY requirements.txt .` then `RUN pip install -r requirements.txt`	install deps **before** copying code, so Docker caches this layer.
A4	`COPY . .`	bring in the rest of the source.

A5	`EXPOSE 8000` + `CMD [...uvicorn... --host 0.0.0.0]`	start the API, reachable from outside the container.

```
# build the image, then run a container from it
docker build -t securenotes .
docker run -p 8000:8000 securenotes      # open http://localhost:8000/docs
```

The two gotchas. Forget `--host 0.0.0.0` and the app listens only *inside* the container — nothing outside can reach it. Copy `. .` before installing and every code edit re-runs the whole `pip install`. Both are exactly what the structural tests check.

Part B — Make it a good image (~25 min)

Task	Do
B1	**Run as non-root.** Create a user and switch to it: `RUN adduser --disabled-password --gecos "" appuser` then `USER appuser`.
B2	**Add a .dockerignore.** Exclude `venv/`, `.git/`, `__pycache__/` and other junk so they never get copied into the image.

Part C — Run the whole stack with Compose (~35 min)

Describe two services in `docker-compose.yml`, then start them together. Check with `pytest tests/test_compose.py`.

Task	Service	Needs
C1	**api**	`build: .` · map port `8000` · `depends_on` the db · pass `DATABASE_URL`.
C2	**db**	`image: postgres:16` · a volume for the data · `POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`.

```
# bring the whole stack up, then prove the containers can talk
docker compose up -d
curl http://localhost:8000/health/db     # -> {"database":"ok"}
docker compose ps                         # api + db both up
docker compose down
```

What proves it works. The `/health/db` endpoint is already in the app. With no database it says `"not configured"`; once compose connects Postgres it returns `"ok"`. That one word is proof the api and db containers are on the same network and talking — the whole point of compose.

Part D — Ship it & submit (~20 min)

```
git init && git add . && git commit -m "Dockerized SecureNotes"
git branch -M main
git remote add origin https://github.com/<you>/securenotes-docker.git
git push -u origin main
```

Submit (via the LMS):

1. GitHub **repo** link (Dockerfile, .dockerignore, docker-compose.yml).
2. **Three screenshots:** the build finishing, `/docs` from the running container, and `docker compose ps` (or the `{"database":"ok"}` response).
3. Completed `TASKS.md` (checklist + reflection).

This is the image your Week 17 pipeline would build and your server would run.

Criteria	Pts
Part A — image builds, serves /docs	**35**
Part B — non-root + .dockerignore + slim pinned base	**20**
Part C — compose runs api + db; /health/db ok	**30**
Part D — repo + proof screenshots	**10**
Reflection answered	**5**